

Answers for Week 5 & 6

1. Amdahl's Law

1. **Autolab** Suppose that a specific task consists of two subtasks. The first subtask retrieves data from a file; it takes 20 seconds to execute on a given system, and it cannot be parallelized. The second subtask processes a large number of data records; it takes 100 seconds to execute sequentially. Each data record can be processed independently, so the second subtask is easily parallelized.

a) What is the parallel fraction of the task consisting of the two subtasks?

Answer: Autolab exercise - answer omitted

b) What is the theoretical speedup for $p = 10$ processors?

Answer: Autolab exercise - answer omitted

c) What is the theoretical maximum speedup?

Answer: Autolab exercise - answer omitted

d) Imagine we currently have 4 processors available for parallization. To improve performance, we have two options:

- i. Optimize the first subtask so we can retrieve the data from the file in 5 seconds instead of 20.
- ii. Purchase more processors so we have 8 processors available instead of 4.

Which option will provide the shortest runtime?

Answer: Autolab exercise - answer omitted

2. Parallel π

The goal of this exercise is to use parallelization to improve the performance of our code. We will use the Monte Carlo approximation of pi from Advanced Python Programming, Chapter 7. Let's take a look at the three different implementations from Advanced Python Programming:

- Implementation 1: Fully serial

```
1 import random
2
3 samples = 1000000
4 hits = 0
5
6 for i in range(samples):
7     x = random.uniform(-1.0, 1.0)
8     y = random.uniform(-1.0, 1.0)
9     if x**2 + y**2 <= 1:
10         hits += 1
11
12 pi = 4.0 * hits/samples
```

- Implementation 2: Fully parallel

```
1 import random
2 import multiprocessing
3
4 def sample():
5     x = random.uniform(-1.0, 1.0)
6     y = random.uniform(-1.0, 1.0)
7     if x**2 + y**2 <= 1:
8         return 1
9     else:
10        return 0
11
12 if __name__ == '__main__':
13     samples = 1000000
14     hits = 0
15
16     n_proc = 10
17     pool = multiprocessing.Pool(n_proc)
18     results_async = [pool.apply_async(sample) for i in range(
19         samples)]
20     hits = sum(r.get() for r in results_async)
21     pi = 4.0 * hits/samples
```

- Implementation 3: Chunked parallel

```
1 import random
2 import multiprocessing
3
```

```

4 def sample():
5     x = random.uniform(-1.0, 1.0)
6     y = random.uniform(-1.0, 1.0)
7     if x**2 + y**2 <= 1:
8         return 1
9     else:
10        return 0
11
12 def sample_multiple(samples_partial):
13     return sum(sample() for i in range(samples_partial))
14
15 if __name__ == '__main__':
16     samples = 1000000
17     hits = 0
18
19     n_proc = 10
20     chunk_size = samples//n_proc
21     pool = multiprocessing.Pool(n_proc)
22     results_async = [pool.apply_async(sample_multiple, (chunk_size
23                                     ,))
24                     for i in range(n_proc)]
25     hits = sum(r.get() for r in results_async)
26     pi = 4.0 * hits/samples

```

1. **Autolab** Run the three implementations as a batch job and measure their run time using the `time` command. Remember to select a CPU model for repeatable results and to select multiple cores.

Answer: Autolab exercise - answer omitted

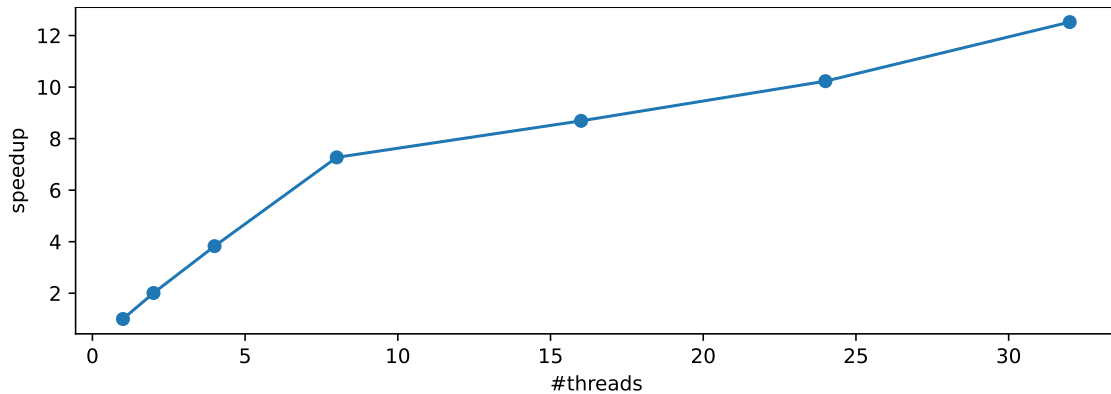
2. **Autolab** Which implementation was fastest? Did some perform slower than expected? Why? Can you relate it to the output of the `'time'` command?

Answer: Autolab exercise - answer omitted

3. Run the fastest parallel implementation for `n_proc` varying from 1 to the maximum number of threads on your CPU. Plot the speedup as a function of the number of processes.

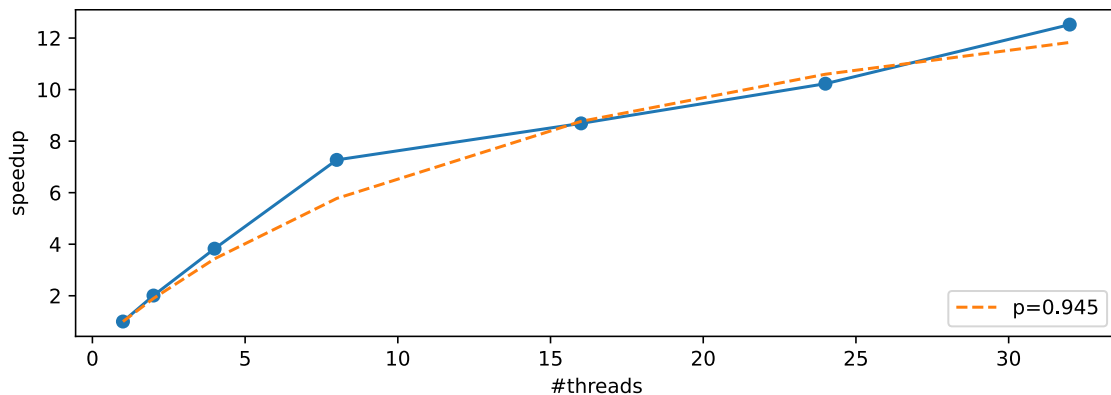
Hint: Use the `lscpu` command to check the number of threads available.

Answer: The exact results will depend on the hardware you run on. Running the implementation on a Xeon Gold 6226R I get the following speedup plot:



4. Estimate the parallel fraction of the program by fitting the theoretical speedup curve from Amdahl's law to your results. Nothing fancy, just play with the numbers until it looks okay. What is the estimated parallel fraction? Given this, what is your theoretical maximum speedup?

Answer: Playing with the fit, I settled on a parallel fraction of 0.945, which looks like this:



The maximum theoretical speedup will then be 18.2.

5. (Optional) Rewrite the code using `pools.map()` instead of `pools.apply_async()` to perform the computation of π . What do you observe? What are pros and cons of `pools.map()`?

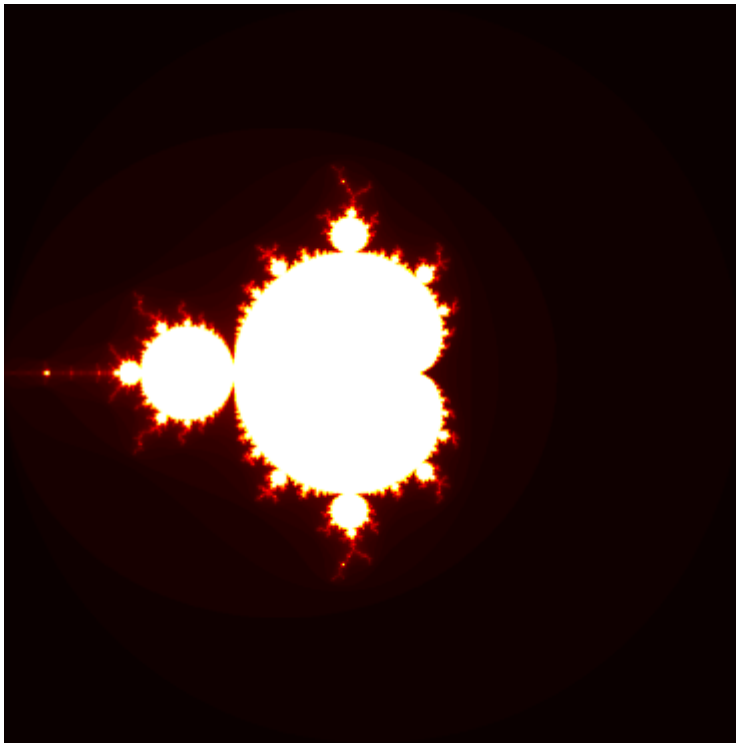
Answer: An easy re-write would be

```
1 import random
```

```
2 import multiprocessing
3 import sys
4
5 def sample(_): # Must take an argument because of map
6     x = random.uniform(-1.0, 1.0)
7     y = random.uniform(-1.0, 1.0)
8     if x**2 + y**2 <= 1:
9         return 1
10    else:
11        return 0
12
13 if __name__ == '__main__':
14     samples = 10000000
15     hits = 0
16     n_tasks = int(sys.argv[1])
17     chunk_size = samples // n_tasks
18     pool = multiprocessing.Pool(n_tasks)
19     res = pool.map(sample, range(samples), chunksize=chunk_size)
20     hits = sum(res)
21     pi = 4.0 * hits/samples
22     print(pi)
```

This is a lot easier to write and understand, which is a pro. However, the performance is not as good as the more "manual" `apply_async` version, since we do so little work per iteration. For tasks with more computation per iteration, the convenience of `map` might be worth it.

3. The Mandelbrot Set



The Mandelbrot Set is a famous mathematical set of complex numbers defined by iterating a simple mathematical function. Given a complex number, c , the iteration is defined by the formula:

$$z_{n+1} = z_n^2 + c.$$

The Mandelbrot Set is the set of complex numbers for which the iteration remains bounded as n approaches infinity. Points outside the set exhibit chaotic and complex behavior when iterated.

In this exercise, we'll explore the computation of the Mandelbrot Set using parallel processing. The task is to generate an image of the Mandelbrot Set by calculating the escape time for each point in the complex plane. The escape time is the number of iterations it takes for a point to achieve an absolute value over 2 - if that happens, it's been shown that it must eventually escape to infinity.

Use the following template for your code:

```
1 import multiprocessing
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def mandelbrot_escape_time(c):
6     z = 0
7     for i in range(100):
```

```

8         z = z**2 + c
9         if np.abs(z) > 2.0:
10             return i
11     return 100
12
13 def generate_mandelbrot_set(points, num_processes):
14     #####
15     # YOUR CODE HERE #
16     #####
17     return escape_times
18
19 def plot_mandelbrot(escape_times):
20     plt.imshow(escape_times, cmap='hot', extent=(-2, 2, -2, 2))
21     plt.axis('off')
22     plt.savefig('mandelbrot.png', bbox_inches='tight', pad_inches=0)
23
24 if __name__ == "__main__":
25     width = 800
26     height = 800
27     xmin, xmax = -2, 2
28     ymin, ymax = -2, 2
29     num_proc = 4
30
31     # Precompute points
32     x_values = np.linspace(xmin, xmax, width)
33     y_values = np.linspace(ymin, ymax, height)
34     points = np.array([complex(x, y) for x in x_values for y in
35                        y_values])
36
37     # Compute set
38     mandelbrot_set = generate_mandelbrot_set(points, num_proc)
39
40     # Save set as image
41     mandelbrot_set = mandelbrot_set.reshape((height, width))
42     plot_mandelbrot(mandelbrot_set)

```

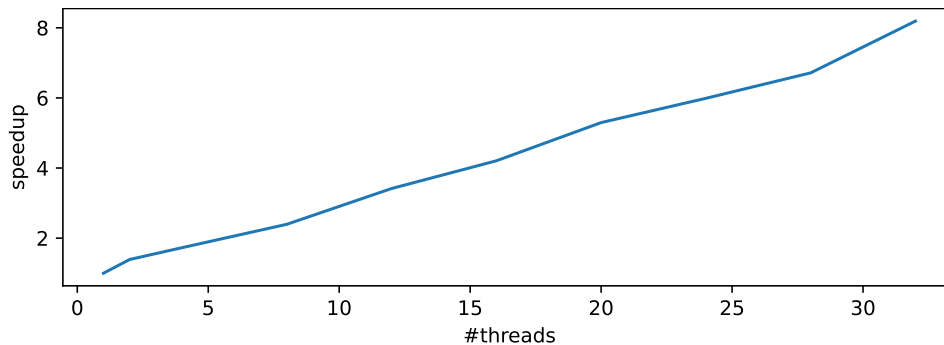
1. **Autolab** Implement the function `generate_mandelbrot_set` to compute the escape times for each complex number in parallel. The function must return a NumPy array. Distribute the points equally between all the processors. Hint: adapt the chunked parallel implementation from the previous exercise.

Answer: Autolab exercise - answer omitted

2. **Autolab** As before, run the program as a batch job where you vary the number of processes `num_proc`. Make a speedup plot.

Answer: Autolab exercise - Job script omitted.

The exact plot will depend on your hardware. Using a Xeon Gold 6226R, I get:



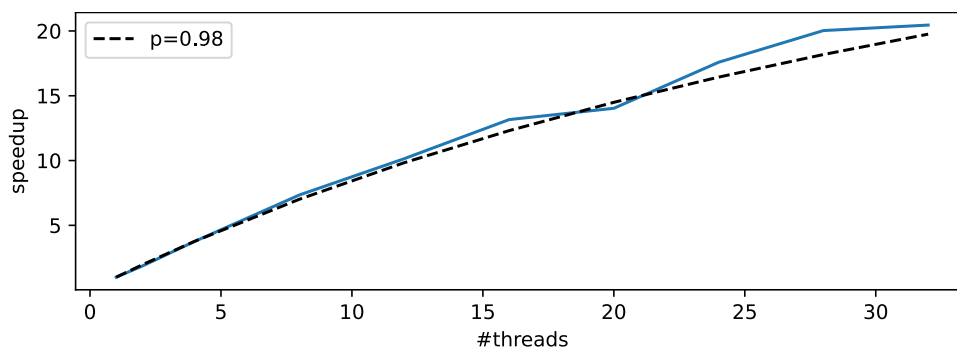
The performance keeps increasing as we add more cores, which is good. However, the speedup is not great considering the amount of hardware we are throwing at the problem. Lets solve that!

-
3. **Autolab** Implement a new function `generate_mandelbrot_set_chunks`, that distributes the points in smaller chunks to the workers. Make sure there are more chunks than workers. Hint: adapt the chunk parallel implementation by setting `chunk_size` to a fixed number.

Answer: Autolab exercise - answer omitted

-
4. Run the new function for varying processes, make a speedup plot and estimate the parallel fraction with Amdahl's law. What do you see? Why does the chunked version achieve a higher speedup? Hint: does all parts of the Mandelbrot set require the same amount of computation?

Answer: Using the improved program, I get the following speedup plot:



We now get a much better speedup and the parallel fraction is now $p = 0.98$ so almost everything is fully parallelized. The maximum theoretical speedup will be 50, which is quite good. The runtime (not shown here) is also improved.

The reason is the time required to compute the escape time varies a lot depending on the location of the point. For the points inside the Mandelbrot set, we must perform all 100 iterations. However, some points will escape almost immediately. When distributing the points evenly at the start, some processes will finish fast and must then wait for the unfortunate ones that got the slow points. Distributing the work in smaller chunks means that processes which finish a chunk quickly can then move on to another chunk. This results in a more even distribution of work and thus a better speedup.
